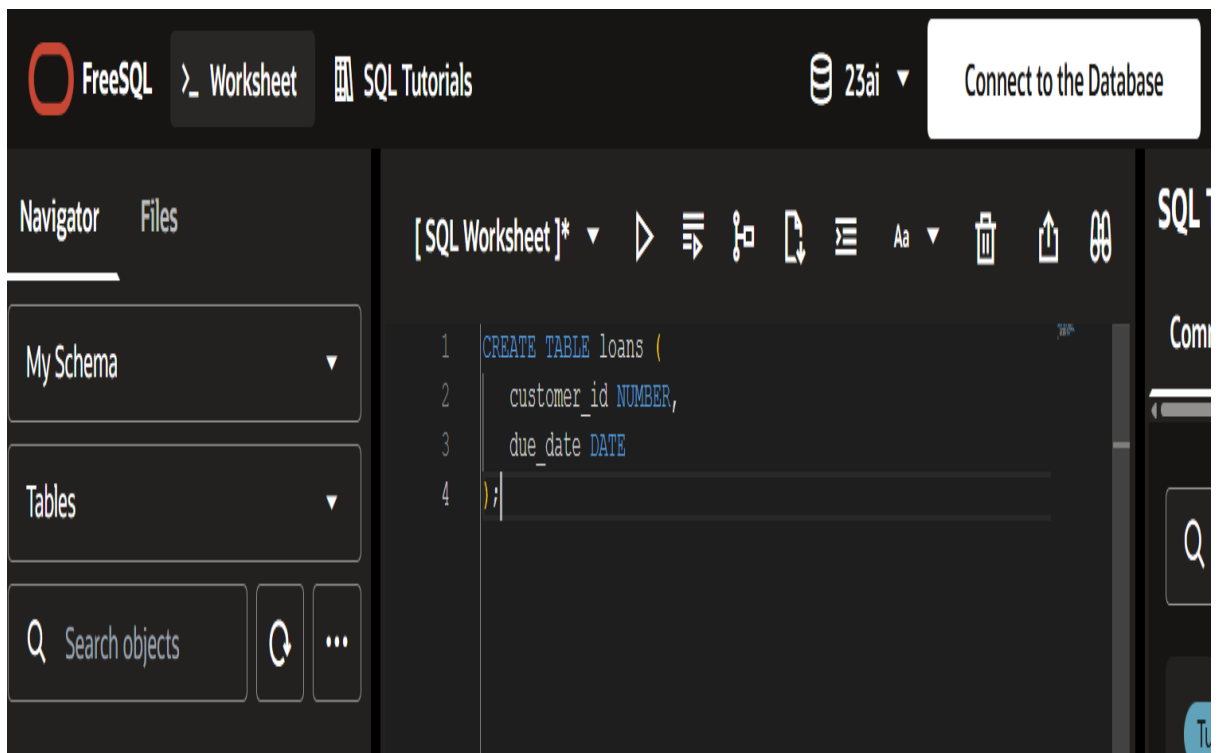
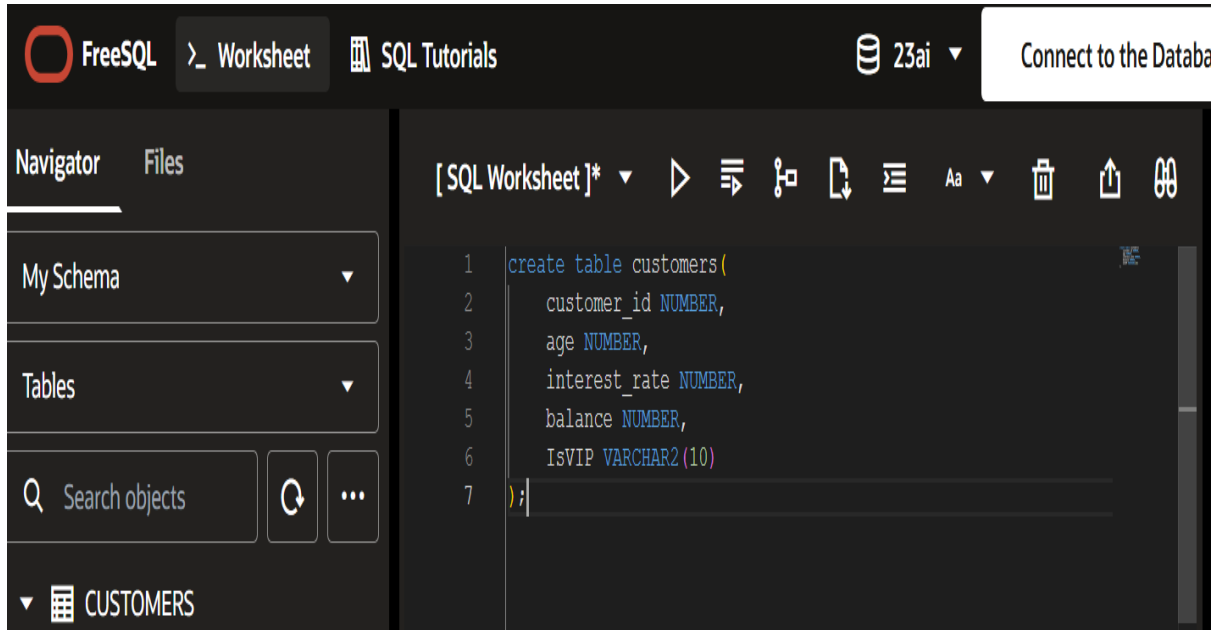
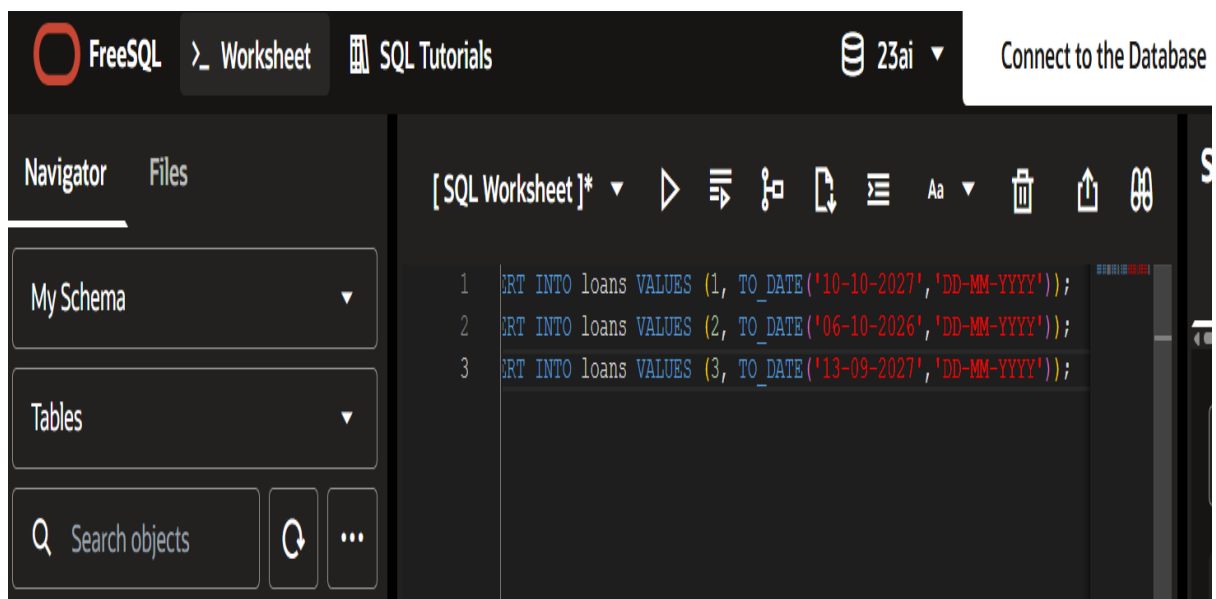
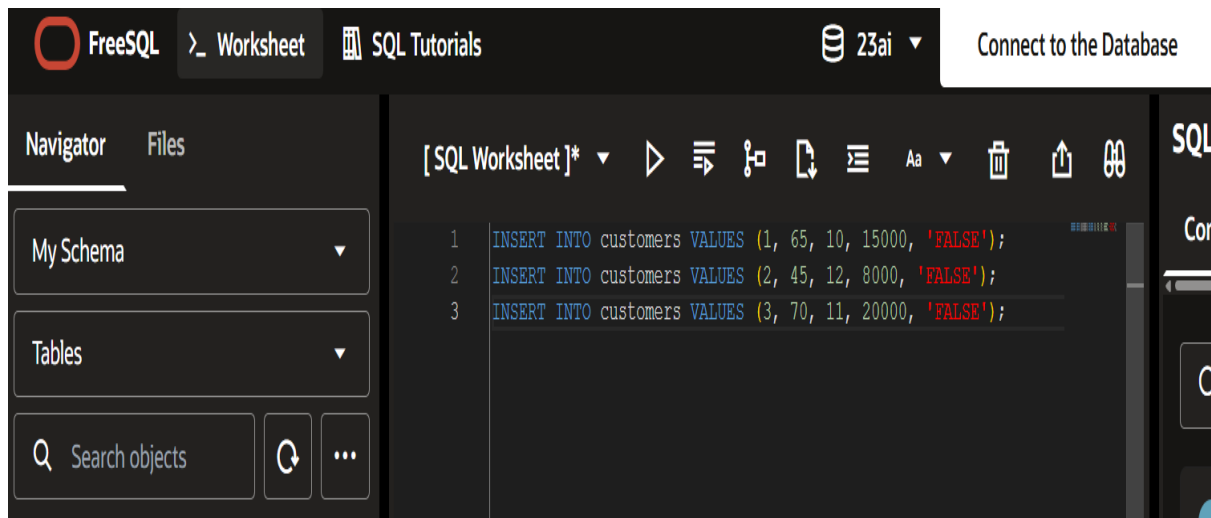


PL/SQL PROGRAMMING

Exercise 1: Control Structures



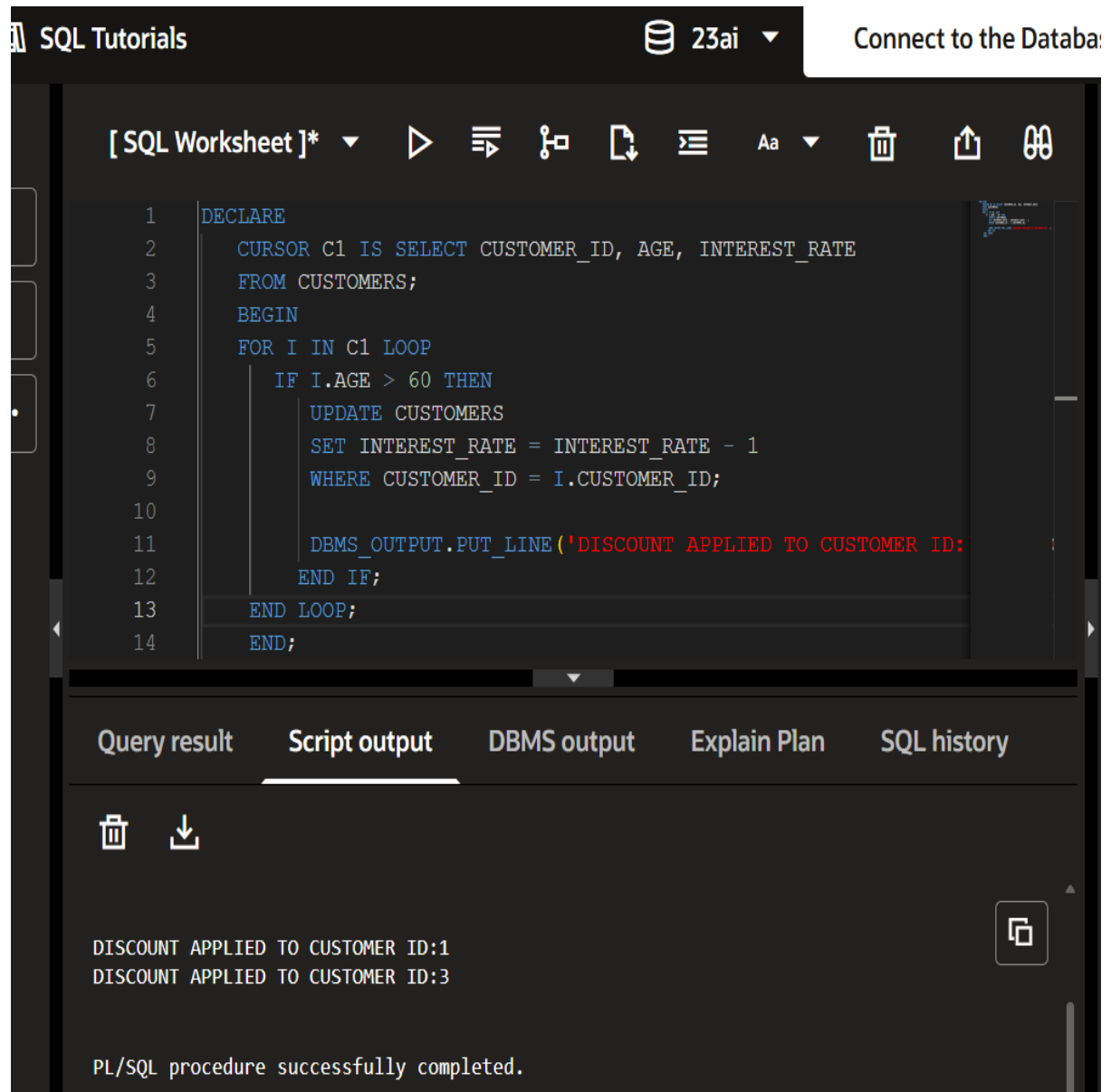


Query result					Script output	DBMS output	Explain Plan	SQL history
  Download ▼ Execution time: 0.156 seconds								
	CUSTOMER_ID	AGE	INTEREST_RATE	BALANCE				
1	1	65	10					
2	2	45	12					
3	3	70	11					

Query result					Script output	DBMS output	Explain Plan	SQL history
  Download ▼ Execution time: 0.156 seconds								
		INTEREST_RATE	BALANCE	ISVIP				
1	65	10	15000	FALSE				
2	45	12	8000	FALSE				
3	70	11	20000	FALSE				

CUSTOMER_ID	DUE_DATE
1	10/10/2027, 12:00:00
2	7/22/2026, 12:00:00
3	9/13/2027, 12:00:00

Question 1: Write a PL/SQL block that loops through all customers, checks their age, and if they are above 60, apply a 1% discount to their current loan interest rates.



The screenshot shows an SQL IDE interface. At the top, there's a header with "SQL Tutorials" and a user profile "23ai". Below the header is a toolbar with various icons for running and editing code. The main area displays a PL/SQL block with the following code:

```
1 DECLARE
2     CURSOR C1 IS SELECT CUSTOMER_ID, AGE, INTEREST_RATE
3     FROM CUSTOMERS;
4 BEGIN
5     FOR I IN C1 LOOP
6         IF I.AGE > 60 THEN
7             UPDATE CUSTOMERS
8             SET INTEREST_RATE = INTEREST_RATE - 1
9             WHERE CUSTOMER_ID = I.CUSTOMER_ID;
10
11             DBMS_OUTPUT.PUT_LINE('DISCOUNT APPLIED TO CUSTOMER ID:
12         END IF;
13     END LOOP;
14 END;
```

Below the code editor, there are tabs for "Query result", "Script output", "DBMS output", "Explain Plan", and "SQL history". The "Script output" tab is active, showing the following output:

```
DISCOUNT APPLIED TO CUSTOMER ID:1
DISCOUNT APPLIED TO CUSTOMER ID:3
```

At the bottom, a message states: "PL/SQL procedure successfully completed."

Question 2: Write a PL/SQL block that iterates through all customers and sets a flag IsVIP to TRUE for those with a balance over \$10,000.

```
1 DECLARE
2     CURSOR C1 IS SELECT CUSTOMER_ID, AGE, INTEREST_RATE, BALANCE, ISVIP
3     FROM CUSTOMERS;
4     BEGIN
5         FOR I IN C1 LOOP
6             IF I.BALANCE > 10000 THEN
7                 UPDATE CUSTOMERS
8                 SET ISVIP = 'TRUE'
9                 WHERE CUSTOMER_ID = I.CUSTOMER_ID;
10
11                 DBMS_OUTPUT.PUT_LINE('ISVIP IS SET TRUE FOR CUSTOMER ID: ' || I.CUSTOMER_ID);
12             END IF;
13         END LOOP;
14     END;
```

Query result **Script output** DBMS output Explain Plan SQL history

ISVIP IS SET TRUE FOR CUSTOMER ID: 1
ISVIP IS SET TRUE FOR CUSTOMER ID: 3

PL/SQL procedure successfully completed.

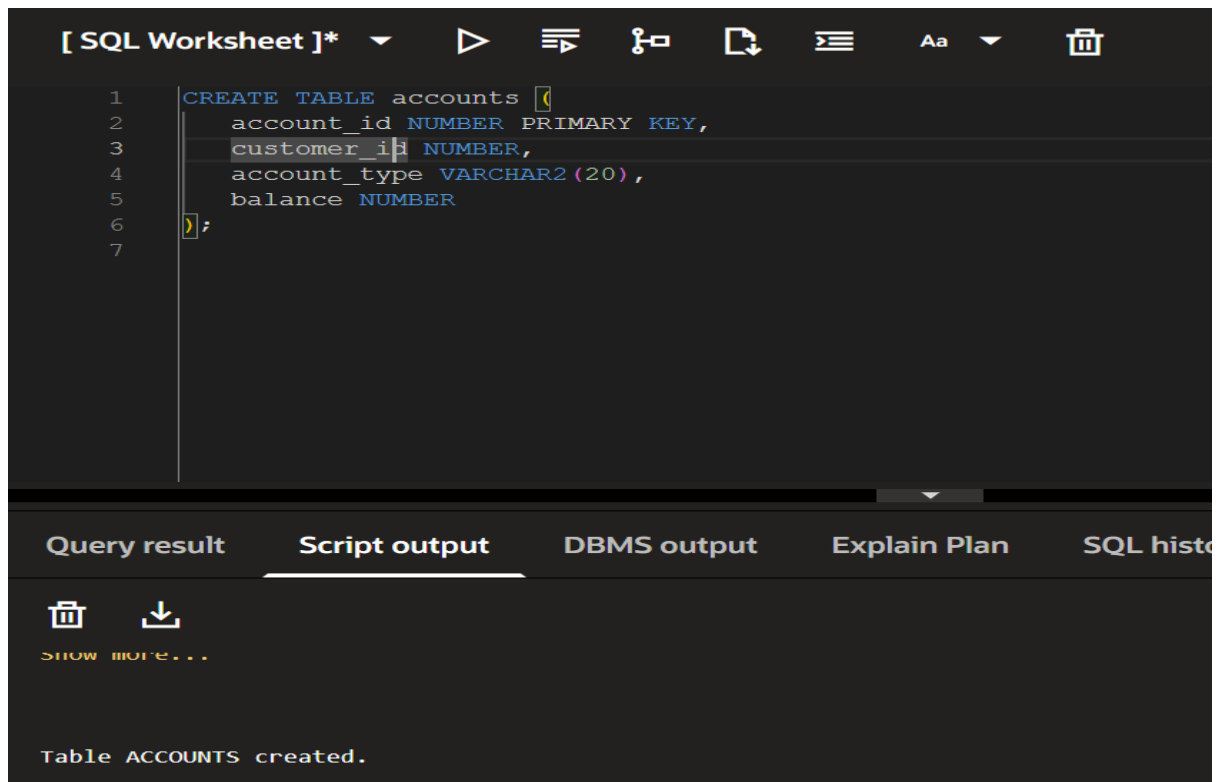
Question 3: Write a PL/SQL block that fetches all loans due in the next 30 days and prints a reminder message for each customer

```
1 DECLARE
2     CURSOR C1 IS SELECT CUSTOMER_ID, DUE_DATE FROM LOANS;
3     BEGIN
4         FOR I IN C1 LOOP
5             IF I.DUE_DATE <= SYSDATE + 30 THEN
6                 DBMS_OUTPUT.PUT_LINE('Reminder: Loan due for Customer ID: ' || I.customer_id ||
7                 ' on ' || I.due_date);
8             END IF;
9         END LOOP;
10     END;
```

Query result **Script output** DBMS output Explain Plan SQL history

Reminder: Loan due for Customer ID: 2 on 22-JUL-26

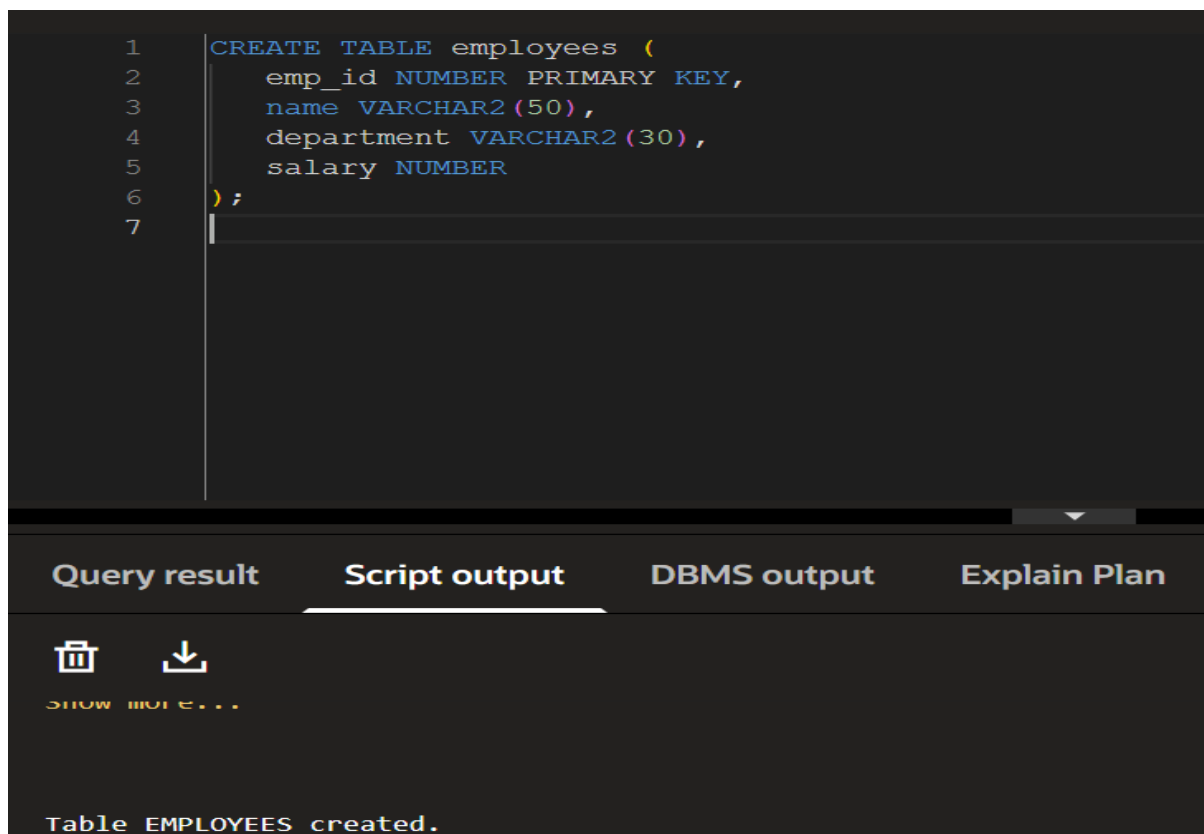
Exercise 3: Stored Procedures



The screenshot shows a SQL Worksheet interface with a dark theme. The top toolbar includes icons for running queries, saving, and other standard SQL tool functions. The main editor area contains the following SQL code:

```
1 CREATE TABLE accounts (  
2     account_id NUMBER PRIMARY KEY,  
3     customer_id NUMBER,  
4     account_type VARCHAR2(20),  
5     balance NUMBER  
6 );  
7
```

Below the editor, the 'Script output' tab is selected, displaying the message: 'Table ACCOUNTS created.'.



The screenshot shows a SQL Worksheet interface with a dark theme. The main editor area contains the following SQL code:

```
1 CREATE TABLE employees (  
2     emp_id NUMBER PRIMARY KEY,  
3     name VARCHAR2(50),  
4     department VARCHAR2(30),  
5     salary NUMBER  
6 );  
7
```

Below the editor, the 'Script output' tab is selected, displaying the message: 'Table EMPLOYEES created.'.

[SQL Worksheet]*

Aa

1

INSERT INTO accounts VALUES (101, 1, 'SAVINGS', 10000);

2

INSERT INTO accounts VALUES (102, 2, 'CURRENT', 5000);

3

INSERT INTO accounts VALUES (103, 3, 'SAVINGS', 20000);

4

[SQL Worksheet]*

Aa

1

INSERT INTO employees VALUES (1, 'John', 'IT', 50000);

2

INSERT INTO employees VALUES (2, 'Alice', 'HR', 40000);

3

INSERT INTO employees VALUES (3, 'Bob', 'IT', 60000);

4

Query result

Script output

DBMS output

Explain Plan

SQL history

Download

Execution time: 0.012 seconds

	ACCOUNT_ID	CUSTOMER_ID	ACCOUNT_TYPE	BALANCE
1	101	1	SAVINGS	10000
2	102	2	CURRENT	5000
3	103	3	SAVINGS	20000

SAVINGS

Download

Execution time: 0.013 seconds

	EMP_ID	NAME	DEPARTMENT	SALARY
1	1	John	IT	50000
2	2	Alice	HR	40000
3	3	Bob	IT	60000

Question 1: Write a stored procedure **ProcessMonthlyInterest** that calculates and updates the balance of all savings accounts by applying an interest rate of 1% to the current balance.

```
1 CREATE OR REPLACE PROCEDURE ProcessMonthlyInterest IS
2 BEGIN
3     UPDATE accounts
4     SET balance = balance + (balance * 0.01)
5     WHERE account_type = 'SAVINGS';
6
7     DBMS_OUTPUT.PUT_LINE('Monthly interest applied to savings accounts');
8 END;
```

Query result Script output DBMS output Explain Plan SQL history

Procedure PROCESSMONTHLYINTEREST compiled

```
0 BEGIN
1     ProcessMonthlyInterest;
2 END;
```

```
Monthly interest applied to savings accounts

PL/SQL procedure successfully completed.
```

```
1 select * from accounts;
```

	ACCOUNT_ID	CUSTOMER_ID	ACCOUNT_TYPE	BALANCE
1	101	1	SAVINGS	10100
2	102	2	CURRENT	5000
3	103	3	SAVINGS	20200

Question 2: Write a stored procedure **UpdateEmployeeBonus** that updates the salary of employees in a given department by adding a bonus percentage passed as a parameter.

```
1 CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus(
2     dept_name VARCHAR2,
3     bonus_percent NUMBER
4 ) IS
5 BEGIN
6     UPDATE employees
7     SET salary = salary + (salary * bonus_percent / 100)
8     WHERE department = dept_name;
9
10    DBMS_OUTPUT.PUT_LINE('Bonus applied to department: ' || dept_name);
11 END;
```

Query result	Script output	DBMS output	Explain Plan	SQL history
<pre>dept_name VARCHAR2, bonus_percent NUMBER) IS...</pre> Show more... Procedure UPDATEEMPLOYEEBONUS compiled				

```
13 BEGIN
14     UpdateEmployeeBonus('IT',10);
15 END;
```

Query result **Script output** DBMS output Explain Plan SQL history

SQL> BEGIN
 UpdateEmployeeBonus('IT',10);
 END;

Bonus applied to department: IT

PL/SQL procedure successfully completed.

Query result Script output DBMS output Explain Plan SQL history					
  Download ▾ Execution time: 0.008 seconds					
	EMP_ID	NAME	DEPARTMENT	SALARY	
1	1	John	IT	55000	
2	2	Alice	HR	40000	
3	3	Bob	IT	66000	

Question 3 : Write a stored procedure **TransferFunds** that transfers a specified amount from one account to another, checking that the source account has sufficient balance before making the transfer

```

1 CREATE OR REPLACE PROCEDURE TransferFunds (
2     from_acc NUMBER,
3     to_acc NUMBER,
4     amount NUMBER
5 ) IS
6     source_balance NUMBER;
7 BEGIN
8     -- Get balance of source account
9     SELECT balance INTO source_balance
10    FROM accounts
11   WHERE account_id = from_acc;
12
13     -- Check sufficient balance
14     IF source_balance >= amount THEN
15
16         -- Deduct from source
17         UPDATE accounts
18        SET balance = balance - amount
19        WHERE account_id = from_acc;
20

```

```

1     -- Add to destination
2     UPDATE accounts
3     SET balance = balance + amount
4     WHERE account_id = to_acc;
5
6     DBMS_OUTPUT.PUT_LINE('Transfer successful');
7
8     ELSE
9         DBMS_OUTPUT.PUT_LINE('Insufficient balance');
10    END IF;
11
12 END;

```

```

34 BEGIN
35     TransferFunds(101,102,2000);
36 END;

```

Query result

Script output

DBMS output

Explain Plan

SQL history



Transfer successful

PL/SQL procedure successfully completed.